Lab7: Cache Coherence Lab

Introduction

In this lab, you need a working understanding of cache coherence protocols and multi-core processor simulation. This lab will guide you through building and analyzing a RISC-V multi-core system with the MOESI (Modified-Exclusive-Shared-Invalid) cache coherence protocol, including:

- **MOESI Protocol Implementation**: Building gem5 with MOESI_CMP_directory cache coherence protocol
- **gem5 Simulator**: Configuring and running multi-core RISC-V systems in gem5
- **Cache Performance Analysis**: Extracting and analyzing cache statistics
- **False Sharing Analysis**: Identifying and optimizing cache coherence issues in multi-threaded programs

These tools and techniques will be used throughout the course for understanding multi-core processor architecture, cache coherence mechanisms, and performance optimization for parallel programs.

Background: gem5 Architecture

Overview of gem5

gem5 is a highly configurable computer system architecture simulator that provides researchers and educators with a powerful platform for studying computer architecture. Originally developed through the merger of the M5 simulator (from the University of Michigan) and GEMS (from the University of Wisconsin), gem5 has become the de facto standard for architectural research and education.

gem5 Cache System Architecture

gem5 incorporates two distinct cache subsystems: "Classic" and "Ruby", inheriting the Classic cache model from its predecessor m5 and the Ruby cache model from GEMS.

1. Classic Cache System

The Classic system is characterized by its ability to rapidly construct simple cache hierarchies with minimal setup effort. However, this simplicity comes at the cost of reduced realism and configurability. For instance, it implements a MOESI snooping-based cache coherence protocol that is tightly coupled with the cache implementation, making it difficult to independently replace or modify the coherence logic.

2. Ruby Cache System

Ruby, conversely, provides a detailed simulation environment for memory subsystems, capable of modeling inclusive or exclusive cache hierarchies, diverse cache replacement policies, cache coherence protocol implementations, interconnection networks, DMA and memory controllers, and sequencers. These models are modular, flexible, and highly configurable, enabling researchers to customize and study individual memory hierarchy components in isolation. Specifically, Ruby supports numerous coherence implementations—including broadcast (snooping), directory-based, token-based, and regional coherence protocols—and can easily be extended to accommodate new coherence models.

SLICC

SLICC (Specification Language including Cache Coherence) is a domain-specific language used in the Ruby memory system for specifying cache coherence protocols. It enables rapid prototyping of new coherence implementations through high-level state machine descriptions, automatically generating the corresponding C++ code for simulation.

gem5 Execution Modes

gem5 supports two primary execution modes, each serving different simulation purposes:

1. Syscall Emulation (SE) Mode

In SE mode, gem5 emulates system calls by trapping them and passing them to the host operating system. This mode supports multi-threaded applications through a static thread mapping mechanism rather than dynamic scheduling. Internally, gem5 lacks an OS-level thread scheduler; instead, when a thread is created via the clone system call (triggered by pthread_create), it is bound to a specific hardware thread context. Each CPU in the simulator contains multiple hardware threads, and every software thread is statically mapped to one of these contexts upon creation. When a new thread is spawned, gem5 locates an available hardware thread, copies the program counter, register values, and state into that context, and activates execution immediately without runtime migration or scheduling decisions.

2. Full-System (FS) Mode

FS mode simulates a complete bare-metal environment capable of running real operating systems. It provides comprehensive support for interrupts, exceptions, privilege levels, and I/O devices, enabling full hardware-software interaction studies.

Step 1: Building MOESI Protocol in gem5

1.1 Compiling RISC-V Programs

Verify Toolchain Installation

```
cd ~/chipyard
source env.sh
riscv64-unknown-linux-gnu-gcc --version
```

Compile Test Programs

For this lab, we'll use a multi-threaded program demonstrating false sharing:

```
riscv64-unknown-linux-gnu-gcc -pthread -static -Wall -o false_sharing_bad_mt
false_sharing_mt.c
```

1.2 Compiling gem5 with MOESI_CMP_directory Protocol

Verify Current Configuration

First, check the current gem5 build configuration:

```
cd /home/gem5
build/RISCV/gem5.opt --build-info
```

Expected Issue: If the protocol shows `MI_example` instead of `MOESI_CMP_directory`, you need to recompile.

Modify Build Configuration

Edit the build options file to use MOESI_CMP_directory protocol:

```
# Edit build_opts/RISCV_MOESI
vi build_opts/RISCV_MOESI
```

Content:

```
RUBY=y
PROTOCOL="MOESI_CMP_directory"
RUBY_PROTOCOL_MOESI_CMP_directory=y
BUILD_ISA=y
USE_RISCV_ISA=y
SLICC_HTML=y
```

Configuration Options Explained:

- `RUBY=y`: Enable Ruby memory system (required for detailed coherence protocols)
- `PROTOCOL`: Specifies the cache coherence protocol to use
- `RUBY_PROTOCOL_MOESI_CMP_directory=y`: Enable the MOESI protocol with directory-based coherence
- `USE_RISCV_ISA=y`: Target RISC-V Instruction Set Architecture
- `SLICC_HTML=y`: Generate HTML documentation for the coherence protocol

Recompile gem5

```
# Clean previous build (if needed)
rm -rf build/RISCV_MOESI

# Rebuild with MOESI protocol. scons is a Python-based build tool and a drop-in
replacement for Make.
scons build/RISCV_MOESI/gem5.opt -j 16
```

Expected Compilation Time: 10-30 minutes depending on your system

Understanding SLICC_HTML=y: When this option is enabled, SLICC generates HTML tables showing the protocol state machines. These can be found in: ./build/RISCV_MOESI/mem/ruby/protocol/html/MOESI_CMP_directory/

By default, SLICC skips HTML generation to improve compilation speed, especially when compiling on network file systems.

Note

For your convenience, we provide a pre-compiled version for Ubuntu 24.04, just like in Lab 1. If you encounter Python issues (e.g., PYTHONHOME =(not set)), you can try to delete libs/libpython3.12.so.1.0 and install the required packages with:

```
sudo apt install python3.12-full python3.12-dev python3.12-venv
```

If you still have problems running it, please contact the TA.

```
tar -xvf gem5_package.tar.gz
cd gem5_package
export LD_LIBRARY_PATH=$(pwd)/libs:$LD_LIBRARY_PATH
```

Verify Compilation

```
build/RISCV_MOESI/gem5.opt --build-info | grep PROTOCOL
```

Expected Output:

```
PROTOCOL = MOESI_CMP_directory
...
RUBY_PROTOCOL_MOESI_CMP_directory = True
```

1.3 Running gem5 with MOESI Protocol

Configuration File

Setup Instructions:

1. Create a new folder named `lab7`
2. Save the Python script below as `riscv_moesi_riscv.py` in the `lab7` directory
3. Ensure the gem5 `configs` directory can be found at `configs_dir = os.path.abspath("../configs")` (this path should resolve correctly from the lab7 folder)

The following Python configuration sets up a RISC-V multi-core system with the MOESI protocol:

```
#!/usr/bin/env python3
"""
riscv_moesi_riscv.py
RISC-V MOESI Cache Coherence Configuration (Timing CPU)
This configuration uses TimingSimpleCPU to demonstrate realistic cache coherence
activity
"""

import argparse
import os
import sys
import shlex

import m5
from m5.objects import *
from m5.defines import buildEnv
from m5.util import addToPath

config_path = os.path.dirname(os.path.abspath(__file__))
configs_dir = os.path.abspath("../configs")

# Ensure configs directory is in sys.path
if configs_dir not in sys.path:
    sys.path.insert(0, configs_dir)

# Use addToPath to add configs directory (points to gem5 root directory)
addToPath(configs_dir)

# Verify path setup
ruby_module_path = os.path.join(configs_dir, 'ruby', 'Ruby.py')
if not os.path.exists(ruby_module_path):
    print(f"ERROR: Cannot find Ruby module at: {ruby_module_path}")
    print(f"Config path: {config_path}")
    print(f"Configs dir: {configs_dir}")
    print(f"Python sys.path:")
    for p in sys.path[:5]:
        print(f"  {p}")
    sys.exit(1)

# Import Ruby module - it will automatically use the compile-time protocol
from ruby.Ruby import create_system, define_options


def main():
    parser = argparse.ArgumentParser(
        description="RISC-V MOESI Cache Coherence with Timing CPU"
    )

    # Add basic options
    parser.add_argument(
        '--num-cpus', type=int, default=2,
```

```
        help='Number of CPU cores (default: 2)'
    )
    parser.add_argument(
        '--benchmark', type=str, required=True,
        help='RISC-V binary to execute (local path)'
    )
    parser.add_argument(
        '--mem-size', type=str, default='32GiB',
        help='Memory size (default: 32GiB, matches DRAM capacity)'
    )
    parser.add_argument(
        '--sys-clock', type=str, default='1GHz',
        help='System clock (default: 1GHz)'
    )
    parser.add_argument(
        '--sys-voltage', type=str, default='1.0V',
        help='System voltage (default: 1.0V)'
    )
    parser.add_argument(
        '--benchmark-args', type=str, default='',
        help='Arguments for benchmark (e.g., "1000")'
    )

    # Add all Ruby-related options
    define_options(parser)

    # Set some default values
    parser.set_defaults(
        cpu_type='TimingSimpleCPU',  # Use RISC-V TimingSimpleCPU
        ruby_clock='1GHz',
        network='simple',
        topology='Crossbar',
        num_dirs=1,
        num_l2caches=2,  # MOESI_CMP_directory protocol requires this parameter
        mem_type='DDR4_2400_16x4',  # Use correct memory type (16x4 configuration)
        enable_dram_powerdown=False,  # DRAM power management option
        l1i_size='32KiB',
        l1d_size='32KiB',
        l2_size='256KiB',
        l1i_assoc=4,
        l1d_assoc=4,
        l2_assoc=8,
        num_l2_banks=0,  # Will be set to num_cpus later
        cacheline_size=64,
        ports=4,
        protocol='MOESI_CMP_directory',
    )

    args = parser.parse_args()

    bench_args = shlex.split(args.benchmark_args) if args.benchmark_args else []

    # Set num_l2_banks default value
    if args.num_l2_banks == 0:
```

```
    args.num_l2_banks = args.num_cpus

# Set num_l2caches (required by MOESI_CMP_directory protocol)
if not hasattr(args, 'num_l2caches') or args.num_l2caches is None:
    args.num_l2caches = args.num_cpus

# Display configuration information
protocol = buildEnv.get('PROTOCOL', 'Unknown')
print(f"\n{'='*60}")
print("RISC-V MOESI Cache Coherence Simulation")
print(f"{'='*60}")
print(f"CPU Cores: {args.num_cpus}")
print(f"Binary: {args.benchmark}")
print(f"Protocol: {protocol}")
print(f"CPU Type: TimingSimpleCPU")
print(f"L1I Cache: {args.l1i_size}, {args.l1i_assoc}-way")
print(f"L1D Cache: {args.l1d_size}, {args.l1d_assoc}-way")
print(f"L2 Cache: {args.l2_size}, {args.l2_assoc}-way")
print(f"Network: {args.network}")
print(f"Topology: {args.topology}")
print(f"{'='*60}\n")

# Create system
system = System()
system.voltage_domain = VoltageDomain(voltage=args.sys_voltage)
system.clk_domain = SrcClockDomain(
    clock=args.sys_clock,
    voltage_domain=system.voltage_domain
)
system.mem_ranges = [AddrRange(args.mem_size)]
system.mem_mode = 'timing'

# Create TimingSimpleCPU instances
system.cpu = [TimingSimpleCPU(cpu_id=i) for i in range(args.num_cpus)]

# Set interrupt controllers and clock domains
for cpu in system.cpu:
    cpu.createInterruptController()
    cpu.clk_domain = system.clk_domain

# Create Ruby system (automatically uses compile-time protocol)
create_system(
    options=args,
    full_system=False,
    system=system,
    dma_ports=[],
    bootmem=None,
    cpus=system.cpu
)

# Set Ruby clock domain
system.ruby.clk_domain = SrcClockDomain(
    clock=args.ruby_clock,
    voltage_domain=system.voltage_domain
```

```
    )

    # Connect CPU ports to Ruby
    for i, ruby_port in enumerate(system.ruby._cpu_ports):
        if ruby_port.support_data_reqs and ruby_port.support_inst_reqs:
            system.cpu[i].icache_port = ruby_port.in_ports
            system.cpu[i].dcache_port = ruby_port.in_ports
        elif ruby_port.support_data_reqs:
            system.cpu[i].dcache_port = ruby_port.in_ports
        elif ruby_port.support_inst_reqs:
            system.cpu[i].icache_port = ruby_port.in_ports

    # Set workload
    if args.benchmark:
        if args.benchmark.startswith("/") or args.benchmark.startswith("./"):
            abs_path = os.path.abspath(args.benchmark)
            process = Process()
            process.cmd = [abs_path] + bench_args

            for cpu in system.cpu:
                cpu.workload = process
                cpu.createThreads()

            system.workload = SEWorkload.init_compatible(abs_path)

    # Create root object
    root = Root(full_system=False, system=system)

    # Set global frequency
    m5.ticks.setGlobalFrequency('1THz')

    # Instantiate simulation
    print("Starting simulation instantiation...")
    m5.instantiate()

    # Run simulation
    print("Starting simulation...")
    exit_event = m5.simulate()

    print(f"\n{'='*60}")
    print("Simulation Complete!")
    print(f"{'='*60}")
    print(f"Exiting @ tick {m5.curTick()} because {exit_event.getCause()}")
    print(f"\nResults can be found in m5out/")
    print(f"\nKey MOESI Cache Coherence Statistics:")
    print(f"  To view all Ruby statistics:")
    print(f"    cat m5out/stats.txt | grep -E 'ruby'")
    print(f"\n")


if __name__ == "__m5_main__":
    main()
```

```c
// false_sharing_mt.c
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <stdatomic.h>

int get_valid_iterations(int argc, char *argv[]) {
    if (argc != 2) {
        fprintf(stderr, "Usage: %s <iterations>\n", argv[0]);
        fprintf(stderr, "Example: %s 1000\n", argv[0]);
        exit(1);
    }

    // Check if parameter is a pure number
    int valid = 1;
    for (int i = 0; argv[1][i] != '\0'; i++) {
        if (argv[1][i] < '0' || argv[1][i] > '9') {
            valid = 0;
            break;
        }
    }

    if (!valid) {
        fprintf(stderr, "Error: Iterations must be a positive integer.\n");
        exit(1);
    }

    int iterations = atoi(argv[1]);

    if (iterations <= 0) {
        fprintf(stderr, "Error: Iterations must be greater than 0.\n");
        exit(1);
    } else if (iterations > 10000000) {
        fprintf(stderr, "Error: Iterations too large, please enter a number not
exceeding 10000000.\n");
        exit(1);
    }

    // printf("Iterations: %d\n", iterations);
    return iterations;
}

/* Structure with false sharing */
struct BadCounter {
    volatile atomic_int counter0;
    volatile atomic_int counter1;  // Shares cache line with counter0
};

struct ThreadData {
    struct BadCounter *counter;
    int thread_id;
    int iterations;
```

```c
};

void* increment_counter(void* arg) {
    struct ThreadData* data = (struct ThreadData*)arg;

    if (data->thread_id == 0) {
        // Thread 0 - access counter0
        for (int i = 0; i < data->iterations; i++) {
            atomic_fetch_add(&data->counter->counter0, 1);
        }
    } else {
        // Thread 1 - access counter1
        for (int i = 0; i < data->iterations; i++) {
            atomic_fetch_add(&data->counter->counter1, 1);
        }
    }

    return NULL;
}

int main(int argc, char *argv[]) {
    int iterations = get_valid_iterations(argc, argv);

    struct BadCounter *counter = malloc(sizeof(struct BadCounter));
    atomic_init(&counter->counter0, 0);
    atomic_init(&counter->counter1, 0);

    pthread_t thread0, thread1;
    struct ThreadData data0, data1;

    data0.counter = counter;
    data0.thread_id = 0;
    data0.iterations = iterations;
    data1.counter = counter;
    data1.thread_id = 1;
    data1.iterations = iterations;

    // Create two threads
    pthread_create(&thread0, NULL, increment_counter, &data0);
    pthread_create(&thread1, NULL, increment_counter, &data1);

    // Wait for threads to complete
    pthread_join(thread0, NULL);
    pthread_join(thread1, NULL);

    printf("counter0: %d, counter1: %d\n", counter->counter0, counter->counter1);
    free(counter);

    return 0;
}
```

Running the Simulation

```
cd ~/gem5/lab7
../build/RISCV_MOESI/gem5.opt ./riscv_moesi_riscv.py \
    --benchmark ./false_sharing_bad_mt \
    --benchmark-args 100000 \
    --mem-size=8GiB \
    --num-cpus 4
```

If you encounter the error src/mem/physical.cc:247: fatal: Could not mmap 34359738368 bytes for range [0:0x800000000]! when running, you can try reducing the --mem-size parameter.

Expected Output:

```
============================================================
RISC-V MOESI Cache Coherence Simulation
============================================================
CPU Cores: 4
Binary: ./false_sharing_bad_mt
Protocol: MOESI_CMP_directory
CPU Type: TimingSimpleCPU
L1I Cache: 32KiB, 4-way
L1D Cache: 32KiB, 4-way
L2 Cache: 256KiB, 8-way
Network: simple
Topology: Crossbar
============================================================

Starting simulation instantiation...
Global frequency set at 1000000000000 ticks per second
warn: No dot file generated. Please install pydot to generate the dot file and
pdf.
src/arch/riscv/isa.cc:321: info: RVV enabled, VLEN = 256 bits, ELEN = 64 bits
src/base/statistics.hh:279: warn: One of the stats is a legacy stat. Legacy stat
is a stat that does not belong to any statistics::Group. Legacy stat is
deprecated.
src/arch/riscv/isa.cc:321: info: RVV enabled, VLEN = 256 bits, ELEN = 64 bits
src/arch/riscv/isa.cc:321: info: RVV enabled, VLEN = 256 bits, ELEN = 64 bits
src/arch/riscv/isa.cc:321: info: RVV enabled, VLEN = 256 bits, ELEN = 64 bits
src/base/statistics.hh:279: warn: One of the stats is a legacy stat. Legacy stat
is a stat that does not belong to any statistics::Group. Legacy stat is
deprecated.
system.remote_gdb: Listening for connections on port 7000
Starting simulation...
src/mem/ruby/system/Sequencer.cc:707: warn: Replacement policy updates recently
became the responsibility of SLICC state machines. Make sure to setMRU() near
callbacks in .sm files!
src/sim/syscall_emul.cc:97: warn: ignoring syscall set_robust_list(...)
      (further warnings will be suppressed)
src/sim/mem_state.cc:443: info: Increasing stack size by one page.
```

```
src/sim/syscall_emul.cc:86: warn: ignoring syscall mprotect(...)
src/sim/syscall_emul.cc:97: warn: ignoring syscall rt_sigaction(...)
      (further warnings will be suppressed)
src/sim/syscall_emul.cc:97: warn: ignoring syscall rt_sigprocmask(...)
      (further warnings will be suppressed)
src/sim/syscall_emul.cc:86: warn: ignoring syscall mprotect(...)
src/sim/power_state.cc:105: warn: PowerState: Already in the requested power
state, request ignored
src/sim/syscall_emul.cc:86: warn: ignoring syscall mprotect(...)
src/sim/syscall_emul.cc:86: warn: ignoring syscall madvise(...)
src/sim/syscall_emul.cc:86: warn: ignoring syscall madvise(...)
counter0: 100000, counter1: 100000

============================================================
Simulation Complete!
============================================================
```

Step 2: Analyzing the result

2.1 Understanding gem5 Output Directory

After running gem5 simulation, all statistics are stored in the `m5out/` directory:

```
ls -lh m5out/
```

Key Files:

- `stats.txt`: Complete simulation statistics (performance, cache, coherence)
- `config.ini`: System configuration in INI format
- `config.json`: System configuration in JSON format

2.2 Extracting Cache Statistics

L1 Cache Statistics

```
# View L1 instruction cache statistics
cat m5out/stats.txt | grep "L1Icache.m_demand"

# View L1 data cache statistics
cat m5out/stats.txt | grep "L1Dcache.m_demand"
```

Understanding the Output:

```
system.ruby.l1_cntrl0.L1Icache.m_demand_hits         6976
system.ruby.l1_cntrl0.L1Icache.m_demand_misses          247
```

```
system.ruby.l1_cntrl0.L1Icache.m_demand_accesses         7223
...
```

Key Metrics:

- `m_demand_hits`: Number of cache hits for demand accesses
- `m_demand_misses`: Number of cache misses for demand accesses
- `m_demand_accesses`: Total demand accesses (hits + misses)

2.3 Observing Cache State Machine Transitions

To observe detailed cache coherence protocol behavior, enable protocol tracing:

```
../build/RISCV_MOESI/gem5.opt --debug-flags=ProtocolTrace \
    ./riscv_moesi_riscv.py \
    --benchmark ./false_sharing_bad_mt \
    --benchmark-args 10 \
    --mem-size=8GiB \
    --num-cpus 4
```

Sample Protocol Trace Output:

```
26063000   0    L1Cache  Writeback_Ack_Data     SI>I      [0x15ac0, line 0x15ac0]
26063000   0  Directory               GETS      I>IS_M    [0x11ac0, line 0x11ac0]
26070000   1    L2Cache      L1_WBCLEANDATA     IW>S      [0x15ac0, line 0x15ac0]
26117000   0  Directory   Memory_Data_Cache    IS_M>IS     [0x11ac0, line 0x11ac0]
26124000   1    L2Cache               Data     IGS>IGS     [0x11ac0, line 0x11ac0]
Directory-0
26131000   0        Seq               Done        >        [0x11af0, line 0x11ac0]
83 cycles
26131000   0    L1Cache               Data      IS>S      [0x11ac0, line 0x11ac0]
26132000   0        Seq              Begin        >        [0x11af4, line 0x11ac0]
IFETCH
```

Understanding Protocol Traces:

Column	Description
Timestamp	Simulation tick when the event occurred
CPU ID	Which processor core initiated the request
Component	Which coherence component handled the event (L1Cache, L2Cache, Directory, Sequencer)
Event	The coherence event being processed (GETS, Writeback_Ack_Data, etc.)

Column	Description
State Transition	Protocol state change (e.g., IS>S means transitioning from IS to S state)
Address	Memory address and cache line involved

Step 3: False Sharing Detection and Optimization

3.1 Understanding False Sharing

False sharing occurs when multiple threads access different variables that happen to reside on the same cache line. Even though the threads are not logically sharing data, the cache coherence protocol treats the accesses as sharing, causing:

- Excessive cache invalidations
- Unnecessary cache-to-cache transfers
- Performance degradation

Example Scenario:

```
struct Counter {
    int counter0;  // Thread 0 accesses this
    int counter1;  // Thread 1 accesses this
};

// Both counters likely in the same 64-byte cache line
// Thread 0's writes invalidate Thread 1's cache line (and vice versa)
```

3.2 False Sharing Test Program

Original Code (with False Sharing):

Compile:

```
riscv64-unknown-linux-gnu-gcc -pthread -static -O2 -Wall -o false_sharing_bad_mt
false_sharing_mt.c
```

Run with False Sharing (Bad Version)

```
cd ~/gem5/lab7
../build/RISCV_MOESI/gem5.opt ./riscv_moesi_riscv.py \
    --benchmark ./false_sharing_bad_mt \
    --benchmark-args 100000 \
    --mem-size=8GiB \
    --num-cpus 4
```

View L1 Data Cache Statistics

```
# Bad version (with false sharing)
cat m5out/stats.txt | grep "L1Dcache.m_demand"
```

Optimized Code (without False Sharing):

You need to modify the code and recompile it.

Compile:

```
riscv64-unknown-linux-gnu-gcc -pthread -static -O2 -Wall -o false_sharing_good_mt
false_sharing_mt.c
```

Run with False Sharing (Good Version)

```
cd ~/gem5/lab7
../build/RISCV_MOESI/gem5.opt ./riscv_moesi_riscv.py \
    --benchmark ./false_sharing_good_mt \
    --benchmark-args 100000 \
    --mem-size=8GiB \
    --num-cpus 4
```

View L1 Data Cache Statistics

```
# Good version (without false sharing)
cat m5out/stats.txt | grep "L1Dcache.m_demand"
```

Laboratory Questions

Please submit the following:

1. gem5 Installation Verification

Screenshot showing successful installation of gem5:

```
cd ~/gem5/lab7

../build/RISCV_MOESI/gem5.opt --build-info | grep PROTOCOL
```

Expected Output:

```
PROTOCOL = MOESI_CMP_directory
...
RUBY_PROTOCOL_MOESI_CMP_directory = True
```

Run whoami in the same terminal after a successful build.

Submit: a screenshot showing the build output and the whoami result

2. Protocol Trace Analysis

Screenshot showing state machine transitions:

Run the simulation with protocol tracing enabled:

```
../build/RISCV_MOESI/gem5.opt --debug-flags=ProtocolTrace \
    ./riscv_moesi_riscv.py \
    --benchmark ./false_sharing_bad_mt \
    --benchmark-args 10 \
    --mem-size=8GiB \
    --num-cpus 4
```

As long as there are state transitions, it's fine. You can exit anytime with Ctrl+C."

Run whoami in the same terminal after a successful build.

Submit:

1. A screenshot showing the state transitions (e.g., I>IS_M, IS>S, M>I) from running the simulation with `--debug-flags=ProtocolTrace`
2. A screenshot showing the whoami result in the same terminal

3. False Sharing Analysis

Please answer the following questions and complete the tasks:

a) Why does false sharing occur?

b) How can false sharing be improved/eliminated?

c) How can false sharing be observed in stats.txt?

Submit:

1. **Answers to the three questions** (a, b, c) above explaining false sharing, how to fix it, and how to detect it in statistics
2. **Your optimized code snippet** showing how you eliminated false sharing
3. **Data analysis** comparing the bad version vs. good version:
    - Screenshots or text showing L1D cache miss rates for both versions
    - Analysis of how the cache statistics demonstrate the improvement
    - Screenshots showing the whoami result in the same terminal

---

References

1. **gem5 Documentation**: https://www.gem5.org/documentation/

2. **XiangShan GEM5 Simulator Documentation**: https://xs-gem5.readthedocs.io/

3. Binkert et al. "The gem5 Simulator." ACM SIGARCH Computer Architecture News, 2011. https://pages.cs.wisc.edu/~karu/courses/cs752/fall2018/papers/gem5.pdf

4. Austin et al. "Efficient Parallel System Simulation with gem5." Workshop on Computer Architecture Research (WACA), 2016. https://www.csl.cornell.edu/~cbatten/pdfs/k-austin-efficient-parallel-sim-waca2016.pdf